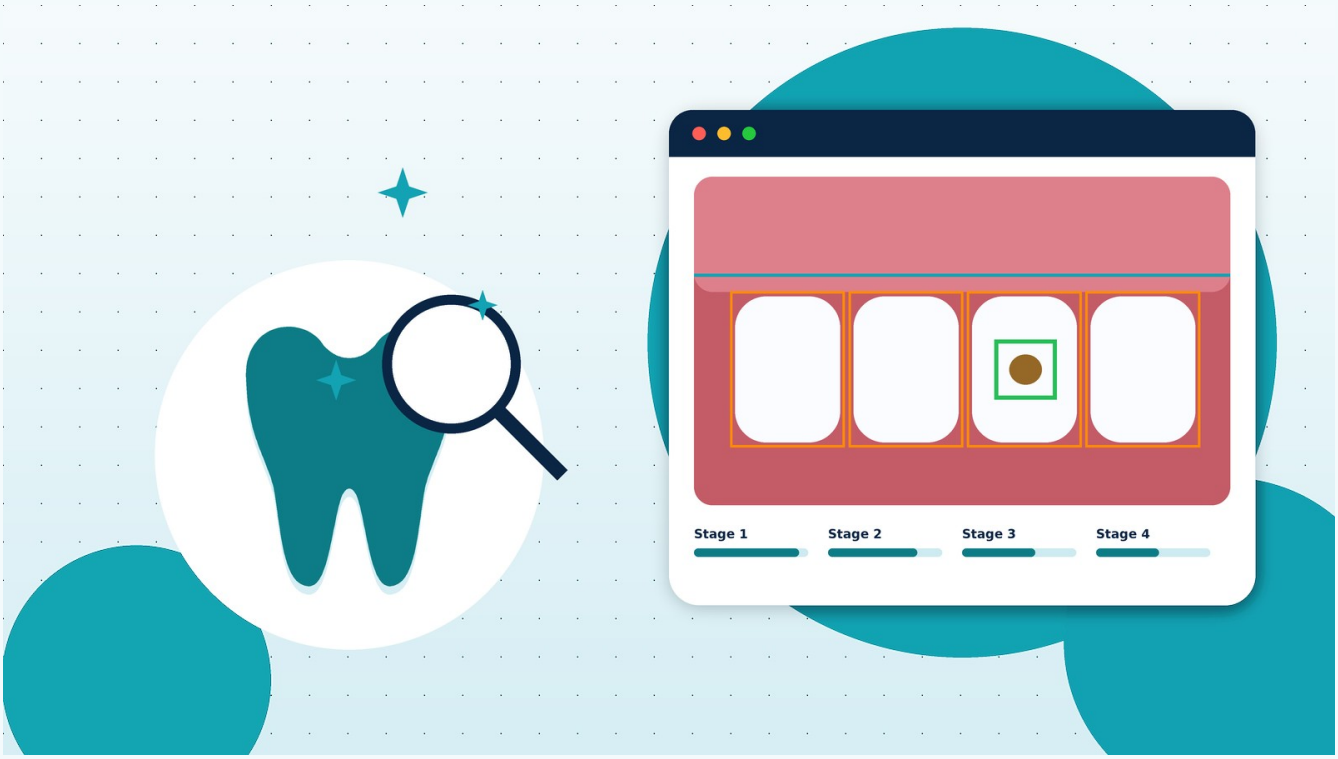
Dental Caries AI Assessment System

Step-by-step guide: running the website in Google Colab

For Dental_Caries_AI_Website.zip (version 1.0.0) · entry point app.py · Colab notebook colab/Run_Dental_Caries_AI_Website.ipynb · 21 October 2026



What this guide covers

- opening Google Colab with the right Google account and reaching the project in Google Drive
- uploading and extracting the website ZIP, installing the requirements
- giving the website the real trained models from pipeline_stage1_to_4_corrected
- launching Streamlit and opening the website link (private Colab proxy or public tunnel)
- running a complete assessment, downloading the report, restarting after a disconnect
- troubleshooting and a final checklist

This website is an academic AI proof of concept. Its outputs are model predictions from a photograph, not a clinical diagnosis, and it is not a substitute for examination by a qualified dentist.

Contents

1. Overview — what you are going to run . . . 3

How the pieces fit together in Colab . . . 3

The trained models used by the website . . . 3

2. Prerequisites . . . 4

Files inside the ZIP (generated from the delivered project) . . . 4

3. Open Google Colab with the right account . . . 5

4. Access the project in Google Drive . . . 5

If you use the account that OWNS the folder . . . 5

If the folder is SHARED with you (e.g. your Gmail account) . . . 5

Upload the website ZIP to Drive . . . 5

5. Run the setup notebook, step by step . . . 7

Step 1 — Check the runtime and mount Google Drive . . . 7

Step 2 — Locate the website ZIP and the trained models . . . 7

Step 3 — Extract the website . . . 8

Step 4 — Install the requirements . . . 8

Step 5 — Provide the trained models and check them . . . 8

Step 6 — Launch the Streamlit app . . . 9

Step 7 — Open the website (choose ONE option) . . . 10

Step 8 — Status, logs and restart . . . 11

Step 9 — Stop the website . . . 11

6. Run a sample assessment . . . 12

7. Complete start-to-finish example (reference result) . . . 18

8. Download the final report . . . 19

9. After a disconnect — restart and keep your work . . . 19

10. Troubleshooting . . . 20

11. Final checklist . . . 21

12. Appendix — what was tested . . . 21

1. Overview — what you are going to run

The website wraps the four-stage pipeline you trained in Dental_Caries_Pipeline_Stage1_to_Stage4_with_FDI_Stage2.ipynb. It does not retrain anything: it loads the saved .keras models and their *_metadata.json files and reuses the notebook's preprocessing, thresholds, class mappings, lesion→tooth association and report logic.



Figure 1 — Assessment workflow: questionnaire, Stage 1 to Stage 4, report.

How the pieces fit together in Colab

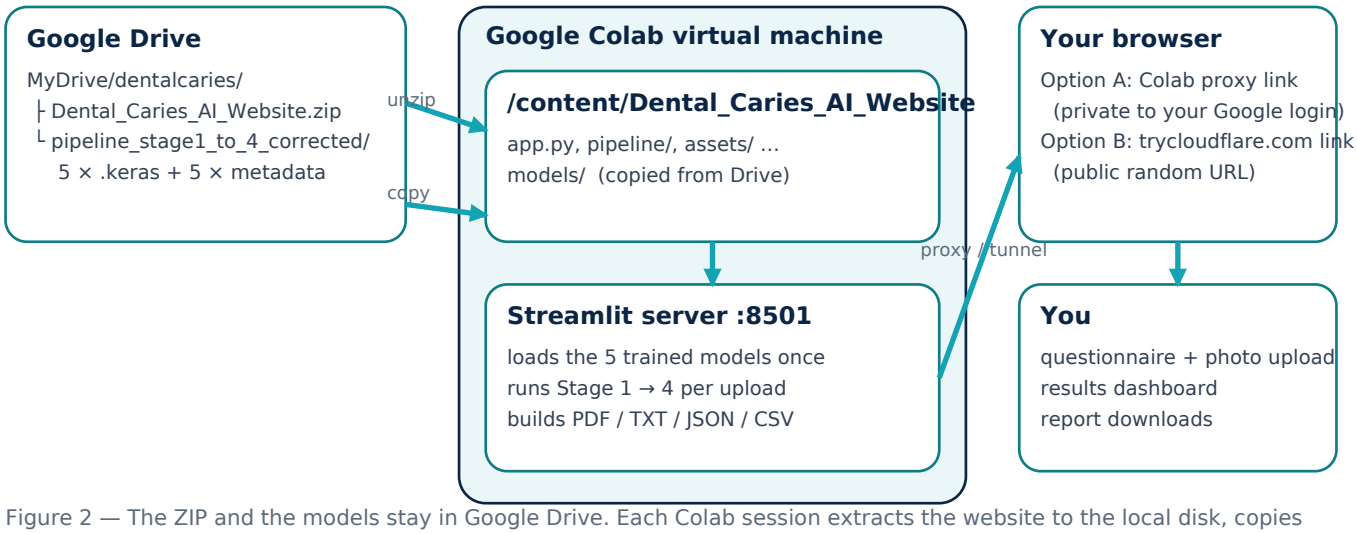


Figure 2 — The ZIP and the models stay in Google Drive. Each Colab session extracts the website to the local disk, copies the models next to it and starts a Streamlit server, which you open through a link.

The trained models used by the website

Stage	File in pipeline_stage1_to_4_corrected/	Size	Settings read from metadata
Stage 1	stage1_caries_model.keras	45.5 MB	EfficientNetB0 · 224 px · threshold 0.603 ± 0.07 · confidence ≥ 0.60
Stage 2	stage2_fdi_detector.keras	29.5 MB	26 FDI classes · tooth score ≥ 0.55 · number confidence ≥ 0.50
Localizer	cavity_localizer_model.keras	29.0 MB	input 416 px · lesion score ≥ 0.40
Stage 3	stage3_gvblack_model.keras	272.0 MB	ResNet50 · G.V. Black Class I–VI · context pad 0.5
Stage 4	stage4_icdas_model.keras	50.4 MB	EfficientNetB0 · ICDAS 1–6 (no ICDAS 0) · window ×1.5

Values as printed by Section 75 of your notebook and stored in the metadata files on Drive. Each .keras file needs its matching *_metadata.json (same name + _metadata).

Important: The models are not inside the ZIP (≈ 426 MB in total). The website reads them from your Drive folder. If a model is missing, its stage is shown as Unavailable — nothing is ever invented or replaced.

2. Prerequisites

- A Google account that can open the Drive folder dentalcaries (owner: your PCCOE account; it is shared with your Gmail account).
- The folder dentalcaries/pipeline_stage1_to_4_corrected with the 5 model files and 5 metadata files (Section 1).
- The file Dental_Caries_AI_Website.zip (this deliverable) on your computer.
- A desktop browser (Chrome, Edge or Firefox). Colab works best on a computer; the website itself also works on a phone.
- About 15 minutes for the first run; about 5 minutes for later runs.
- No API keys, tokens or paid services are needed.

Files inside the ZIP (generated from the delivered project)

Dental_Caries_AI_Website/

```
.streamlit/ config.toml
README.md
app.py
assets/     clinical_caries_example.jpg, favicon.png, fonts, hero_dental_ai.jpg, icons,
            logo.png, tooth_anatomy.png, workflow_stages.png
colab/      Run_Dental_Caries_AI_Website.ipynb
components/ __init__.py, image_viewer.py, questionnaire.py, result_cards.py, styles.py
config.py
docs/       Dental_Caries_AI_Colab_Guide.pdf, TEST_REPORT.md, images, screenshots
models/     README.md
pipeline/   __init__.py, annotation.py, image_validation.py, lesion_localization.py,
            model_loader.py, preprocessing.py, questionnaire.py, report_generator.py,
            runner.py, stage1.py, stage2_fdi.py, stage3_gvblack.py, stage4_icdas.py
reference/  Dental_Caries_Pipeline_Stage1_to_Stage4_with_FDI_Stage2.ipynb
requirements-colab.txt
requirements-dev.txt
requirements.txt
scripts/    check_models.py
tests/      conftest.py, e2e_browser_check.py, standin_models.py, test_app.py,
            test_notebook_parity.py, test_pipeline.py, test_questionnaire.py,
            test_reports.py, test_validation.py
tools/      colab_cells.py, make_assets.py, make_colab_notebook.py, make_guide.py
```

Tip: Keep the ZIP in Drive. Do not extract it inside Drive — Colab extracts it to its own fast disk in Step 3.

3. Open Google Colab with the right account

1. Go to https://colab.research.google.com.
2. Look at the round profile picture in the top-right corner. Click it and choose the account that can see dentalcaries (the PCCOE account that owns it, or the Gmail account it is shared with). Colab and Drive must use the same account — Drive is mounted as whoever is signed in to Colab.
3. Open the setup notebook: File → Upload notebook → choose colab/Run_Dental_Caries_AI_Website.ipynb from the extracted ZIP on your computer. (Alternative: File → New notebook and paste the code cells from this guide one by one.)
4. Choose a GPU: Runtime → Change runtime type → T4 GPU → Save. A CPU runtime also works, but loading the models and analysing a photo are slower.

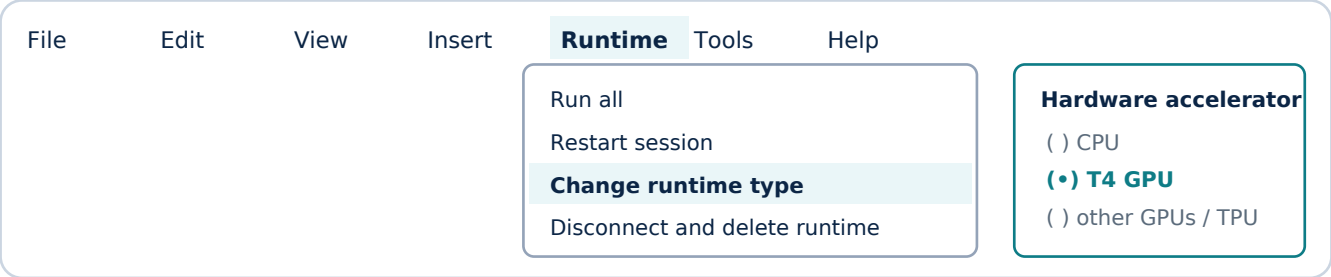


Figure 3 — Runtime → Change runtime type → T4 GPU (schematic).

4. Access the project in Google Drive

The notebook expects the project at MyDrive/dentalcaries, exactly like your training notebook (DRIVE_BASE = "/content/drive/MyDrive/dentalcaries").

If you use the account that OWNS the folder

Nothing to do — it is already at My Drive → dentalcaries.

If the folder is SHARED with you (e.g. your Gmail account)

Shared folders are not inside “My Drive”, so Colab cannot see them until you add a shortcut:

1. Open https://drive.google.com with the same account as Colab.
2. Click Shared with me, right-click the folder dentalcaries.
3. Choose Organize → Add shortcut, select My Drive, click Add.
4. Check: My Drive now shows dentalcaries with a small arrow icon. Colab sees it as /content/drive/MyDrive/dentalcaries.

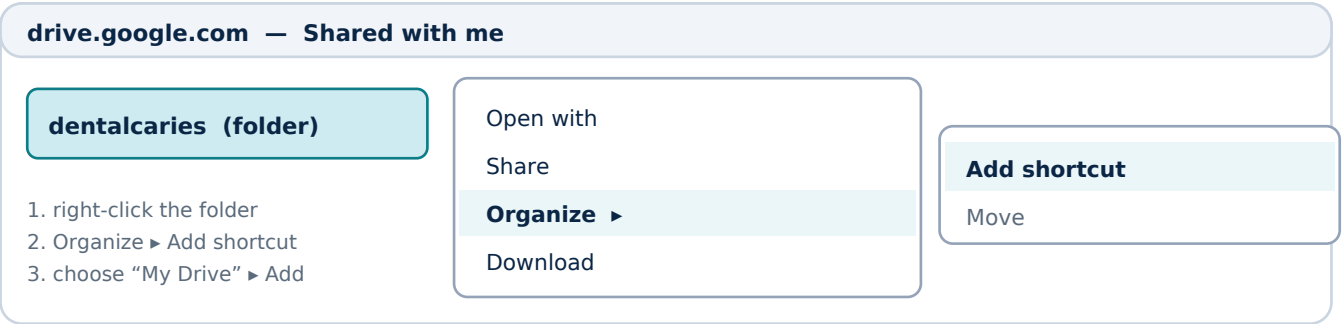


Figure 4 — Adding a shortcut to a shared folder (schematic).

Tip: If you only have view access to the shared folder you cannot upload the ZIP into it. Upload the ZIP to My Drive (root) instead — the notebook also looks there.

Upload the website ZIP to Drive

1. In Drive open My Drive → dentalcaries.
2. Click + New → File upload and choose Dental_Caries_AI_Website.zip.
3. Wait until the upload panel says “1 upload complete”. The file name must stay exactly Dental_Caries_AI_Website.zip (rename it if the browser added “(1)”).

5. Run the setup notebook, step by step

Run each cell with the ▶ button (or Shift+Enter) and wait until it finishes before starting the next one. The cells below are identical to the ones in colab/Run_Dental_Caries_AI_Website.ipynb.

Step 1 — Check the runtime and mount Google Drive

Code cell — copy into Colab

```
# STEP 1 — Check the runtime and mount Google Drive
import os, subprocess
gpu = subprocess.run(["nvidia-smi", "-L"], capture_output=True, text=True).stdout
print(gpu or "No GPU (CPU runtime) - the app still works, just slower")
from google.colab import drive
drive.mount("/content/drive")
print("Drive mounted:", os.path.isdir("/content/drive/MyDrive"))
```

A pop-up asks for permission to access Google Drive: choose the same account as in Section 3 and allow access.

Expected output (values such as sizes and versions may differ slightly)

```
GPU 0: Tesla T4 (UUID: GPU-…)
Mounted at /content/drive
Drive mounted: True
```

Step 2 — Locate the website ZIP and the trained models

Code cell — copy into Colab

```
# STEP 2 — Locate the website ZIP and the trained models on Google Drive
import os, glob
DRIVE_PROJECT = "/content/drive/MyDrive/dentalcaries"              # project folder (or shortcut)
MODELS_DRIVE  = f"{DRIVE_PROJECT}/pipeline_stage1_to_4_corrected"  # written by the notebook
ZIP_CANDIDATES = [f"{DRIVE_PROJECT}/Dental_Caries_AI_Website.zip",
                  "/content/drive/MyDrive/Dental_Caries_AI_Website.zip",
                  "/content/Dental_Caries_AI_Website.zip"]
ZIP_PATH = next((p for p in ZIP_CANDIDATES if os.path.isfile(p)), None)
print("Website ZIP :", ZIP_PATH or "NOT FOUND - upload Dental_Caries_AI_Website.zip first")
print("Model folder:", MODELS_DRIVE, "->", "found" if os.path.isdir(MODELS_DRIVE) else "NOT FOUND")
for f in sorted(glob.glob(f"{MODELS_DRIVE}/*.keras")):
    print(f"   {os.path.basename(f):34} {os.path.getsize(f)/1e6:7.1f} MB")
```

Expected output (values such as sizes and versions may differ slightly)

```
Website ZIP : /content/drive/MyDrive/dentalcaries/Dental_Caries_AI_Website.zip
Model folder: /content/drive/MyDrive/dentalcaries/pipeline_stage1_to_4_corrected -> found
   cavity_localizer_model.keras          29.0 MB
   stage1_caries_model.keras             45.5 MB
   stage2_fdi_detector.keras             29.5 MB
   stage3_gvblack_model.keras           272.0 MB
   stage4_icdas_model.keras              50.4 MB
```

Important: If you see NOT FOUND, fix the Drive location (Section 4) and run Step 2 again. If your models live in a different folder, change MODELS_DRIVE in this cell.

Step 3 — Extract the website

Code cell — copy into Colab

```
# STEP 3 — Extract the website to the fast local Colab disk
import os, zipfile, shutil
APP_DIR = "/content/Dental_Caries_AI_Website"
assert ZIP_PATH, "Website ZIP not found - upload it to MyDrive/dentalcaries/ and run Step 2 again"
shutil.rmtree(APP_DIR, ignore_errors=True)
with zipfile.ZipFile(ZIP_PATH) as z:
    bad = z.testzip()
    assert bad is None, f"The ZIP is damaged (first bad file: {bad}) - upload it again"
    z.extractall("/content")
for f in ["app.py", "config.py", "requirements-colab.txt", "pipeline/runner.py", "assets/logo.png"]:
    print(("OK      " if os.path.exists(f"{APP_DIR}/{f}") else "MISSING ") + f)
%cd /content/Dental_Caries_AI_Website
```

Expected output (values such as sizes and versions may differ slightly)

```
OK      app.py
OK      config.py
OK      requirements-colab.txt
OK      pipeline/runner.py
OK      assets/logo.png
/content/Dental_Caries_AI_Website
```

Step 4 — Install the requirements

Code cell — copy into Colab

```
# STEP 4 — Install the two extra packages (TensorFlow/Keras are already in Colab)
!pip install -q -r /content/Dental_Caries_AI_Website/requirements-colab.txt
import tensorflow as tf, keras, streamlit, reportlab
print("TensorFlow", tf.__version__, "| Keras", keras.__version__)
print("Streamlit", streamlit.__version__, "| ReportLab", reportlab.Version)
```

Only streamlit and reportlab are installed. TensorFlow, Keras, NumPy, pandas, OpenCV, Pillow and Plotly come with Colab. Do not reinstall TensorFlow: your models were trained and saved with Colab's own TensorFlow 2.21.0 / Keras 3.13.2.

Expected output (values such as sizes and versions may differ slightly)

```
TensorFlow 2.21.0 | Keras 3.13.2
Streamlit 1.x.x | ReportLab 4.x
```

Step 5 — Provide the trained models and check them

Code cell — copy into Colab

```
# STEP 5 — Copy the trained models next to the app and check that they load
import os, shutil, glob
LOCAL_MODELS = "/content/Dental_Caries_AI_Website/models"
os.makedirs(LOCAL_MODELS, exist_ok=True)
for f in glob.glob(f"{MODELS_DRIVE}/*.keras") + glob.glob(f"{MODELS_DRIVE}/*_metadata.json"):
    dst = f"{LOCAL_MODELS}/{os.path.basename(f)}"
    if not (os.path.exists(dst) and os.path.getsize(dst) == os.path.getsize(f)):
        shutil.copy(f, dst); print("copied", os.path.basename(f))
os.environ["DCAI_MODELS_DIR"] = LOCAL_MODELS
!cd /content/Dental_Caries_AI_Website && python scripts/check_models.py "$DCAI_MODELS_DIR"
```

The ten files are copied from Drive to /content/Dental_Caries_AI_Website/models (about 1–3 minutes the first time; skipped later in the same session). scripts/check_models.py then loads every model exactly like the website does.

Expected output (values such as sizes and versions may differ slightly)

```
Model folder: /content/Dental_Caries_AI_Website/models
  OK   stage1_caries_model.keras  (45.5 MB)
  OK   stage1_caries_model_metadata.json  (0.0 MB)
  … (all 10 files OK)

Loading models (this can take 1–2 minutes)…

TensorFlow 2.21.0 | Keras 3.13.2
  [Loaded     ] Stage 1 — Healthy vs Caries (CNN 1)
                EfficientNetB0 · 224 px · threshold 0.603 ± 0.07 · confidence ≥ 0.60
  [Loaded     ] Stage 2 — FDI tooth detector
                26 FDI classes · score ≥ 0.55 · number confidence ≥ 0.50
  [Loaded     ] Cavity / lesion localizer      input 416 px · score ≥ 0.40
  [Loaded     ] Stage 3 — G.V. Black classifier (CNN 2)   ResNet50 · … · pad 0.5
  [Loaded     ] Stage 4 — ICDAS classifier (CNN 3)   EfficientNetB0 · … · scores [1, … 6]

RESULT: all required models loaded
(the real script prints each model on one line — the layout above is wrapped to fit the page)
```

Step 6 — Launch the Streamlit app

Code cell — copy into Colab

```
# STEP 6 — Start the Streamlit website in the background
import subprocess, time, urllib.request, os
subprocess.run("pkill -f 'streamlit run' || true", shell=True)
log = open("/content/streamlit.log", "w")
cmd = ["streamlit", "run", "app.py", "--server.port", "8501", "--server.address", "0.0.0.0",
       "--server.headless", "true", "--server.enableCORS", "false",
       "--server.enableXsrfProtection", "false"]
env = {**os.environ, "DCAI_MODELS_DIR": "/content/Dental_Caries_AI_Website/models"}
proc = subprocess.Popen(cmd, cwd="/content/Dental_Caries_AI_Website", env=env,
                        stdout=log, stderr=subprocess.STDOUT)
HEALTH = "http://localhost:8501/_stcore/health"
for _ in range(60):
    try:
        if urllib.request.urlopen(HEALTH, timeout=2).read() == b"ok":
            print("Streamlit is running on port 8501"); break
    except Exception:
        time.sleep(1)
else:
    print("Streamlit did not start - see the log:")
    print(open("/content/streamlit.log").read()[-3000:])
```

Streamlit runs in the background, so the notebook stays usable. The server listens on port 8501; its log is written to /content/streamlit.log. The flags enableCORS false and enableXsrfProtection false are needed because the page is reached through a proxy or tunnel (otherwise photo uploads can fail with error 403).

Expected output (values such as sizes and versions may differ slightly)

```
Streamlit is running on port 8501
```

Important: Streamlit does not become reachable from the internet by itself — Colab machines have no public address. Use Option A or Option B in Step 7 to open it.

Step 7 — Open the website (choose ONE option)

Option A — Colab's built-in proxy (recommended, private). Google serves port 8501 through a link that only works for your signed-in Google account in this browser. No extra software, no account, nothing public.

Code cell — copy into Colab

```
# STEP 7 (Option A, recommended) — Open the website through Colab's private port proxy
from google.colab.output import eval_js
url = eval_js("google.colab.kernel.proxyPort(8501, {'cache': false})")
print("Open this link in the SAME browser where you are signed in to Colab:")
print(url)
```

Expected output (values such as sizes and versions may differ slightly)

```
Open this link in the SAME browser where you are signed in to Colab:
https://…-8501-…colab… (a long Google address)
```

Option B — Cloudflare quick tunnel (fallback, public link). Use it if the Option A page stays blank or keeps reconnecting, or when somebody else must open the website (e.g. an examiner on another computer). It downloads Cloudflare's open-source cloudflared program and creates a temporary random trycloudflare.com address. No account or token is needed and none is stored in the notebook.

Code cell — copy into Colab

```
# STEP 7 (Option B, fallback) — Public HTTPS link with a Cloudflare quick tunnel (no account needed)
# WARNING: anyone who has this random link can open the website while this Colab session runs.
import subprocess, re, time, os
CF = "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64"
if not os.path.exists("/content/cloudflared"):
    !wget -q -O /content/cloudflared {CF}
    !chmod +x /content/cloudflared
tlog = open("/content/cloudflared.log", "w")
tunnel = subprocess.Popen(["/content/cloudflared", "tunnel", "--no-autoupdate",
                           "--url", "http://localhost:8501"], stdout=tlog, stderr=subprocess.STDOUT)
for _ in range(40):
    time.sleep(1)
    text = open("/content/cloudflared.log").read()
    m = re.search(r"https://[a-z0-9-]+\.trycloudflare\.com", text)
    if m:
        print("Website URL:", m.group(0)); break
else:
    print("No URL yet - see /content/cloudflared.log")
```

Expected output (values such as sizes and versions may differ slightly)

```
Website URL: https://random-words-1234.trycloudflare.com
```

Security: A tunnel URL is public: anyone who knows it can use the website (and upload photos to your Colab session) while the session runs. Share it only with people who need it, never post it, and stop it with Step 9 when you are finished. The link changes every time and stops working when the session ends. Quick tunnels have no uptime guarantee and are meant for testing/demos. (Alternative: ngrok with your own free auth token stored in Colab's Secrets panel — never paste a token into a shared notebook.)

Step 8 — Status, logs and restart

Code cell — copy into Colab

```
# STEP 8 — Status, logs and restart (run whenever something looks wrong)
import urllib.request
try:
    r = urllib.request.urlopen("http://localhost:8501/_stcore/health", timeout=3)
    print("Health:", r.read().decode())
except Exception as e:
    print("Streamlit is NOT running:", e, "-> run Step 6 again")
print("---- last lines of /content/streamlit.log ----")
print(open("/content/streamlit.log").read()[-2500:])
```

Expected output (values such as sizes and versions may differ slightly)

```
Health: ok
---- last lines of /content/streamlit.log ----
  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
```

Step 9 — Stop the website

Code cell — copy into Colab

```
# STEP 9 — Stop the website (and the tunnel, if used)
!pkill -f "streamlit run" ; pkill -f cloudflared ; echo "stopped"
```

Expected output (values such as sizes and versions may differ slightly)

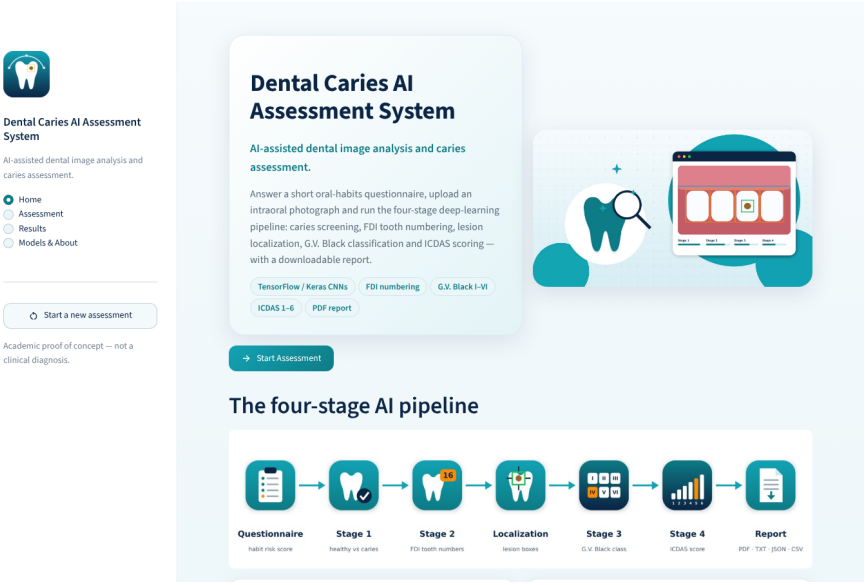
```
stopped
```

6. Run a sample assessment

Important: The screenshots in this section were taken from the real website during automated testing. Because the trained weights could not be downloaded into the test environment, they were captured with test-only stand-in models on a synthetic test image. The layout is exactly what you will see; the predictions shown here are not real. Section 7 shows the real reference result from your notebook.

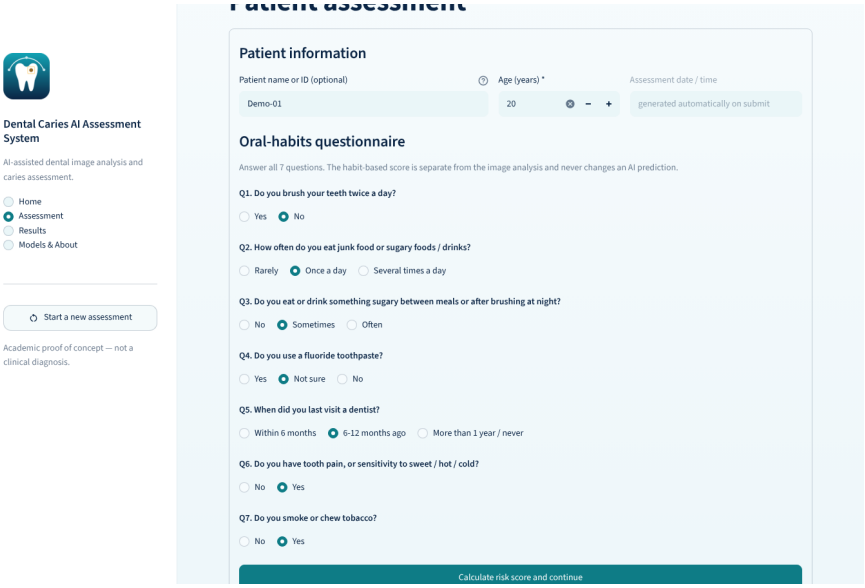
6.1 Home page

The home page with the hero image and the Start Assessment button. Click Start Assessment (or Assessment in the left menu).



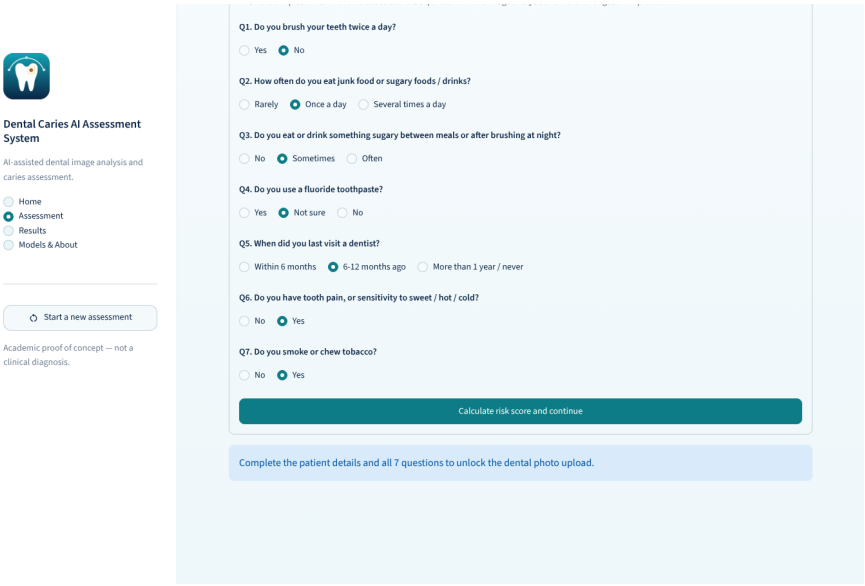
6.2 Patient details

Enter an optional name/ID and the age (required, 1–120). The date and time are recorded automatically when you submit.



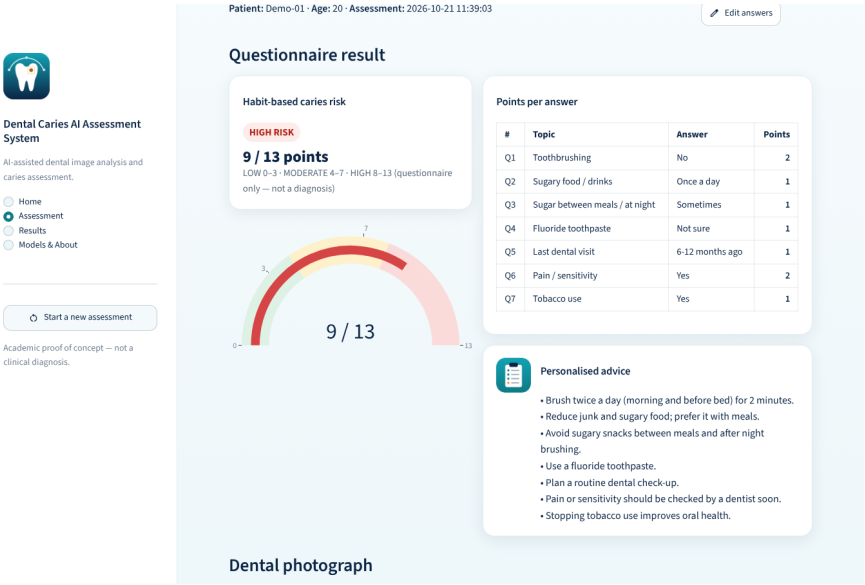
6.3 Questionnaire

Answer all seven questions, then click Calculate risk score and continue. Missing answers or an invalid age are reported in red.



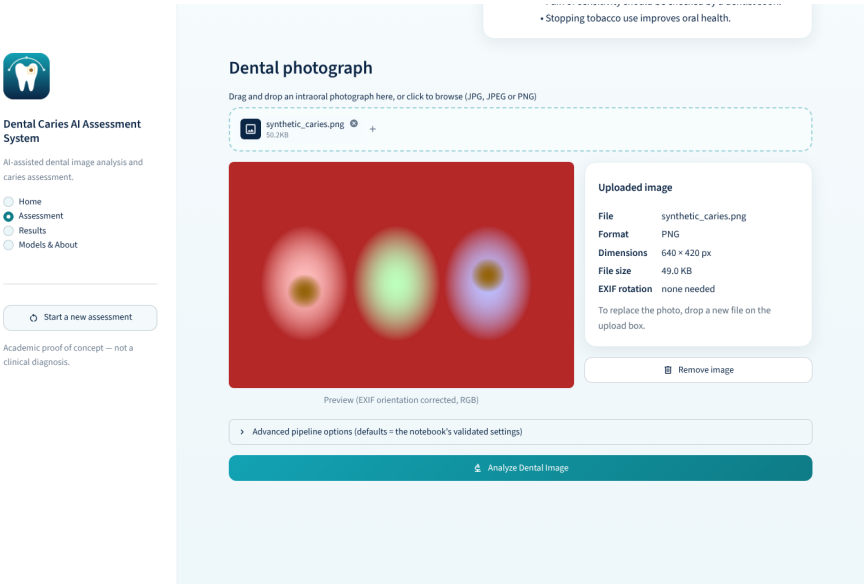
6.4 Questionnaire result

Score out of 13, risk category (LOW 0–3, MODERATE 4–7, HIGH 8–13), points per answer and personalised advice. Click Edit answers to change them.



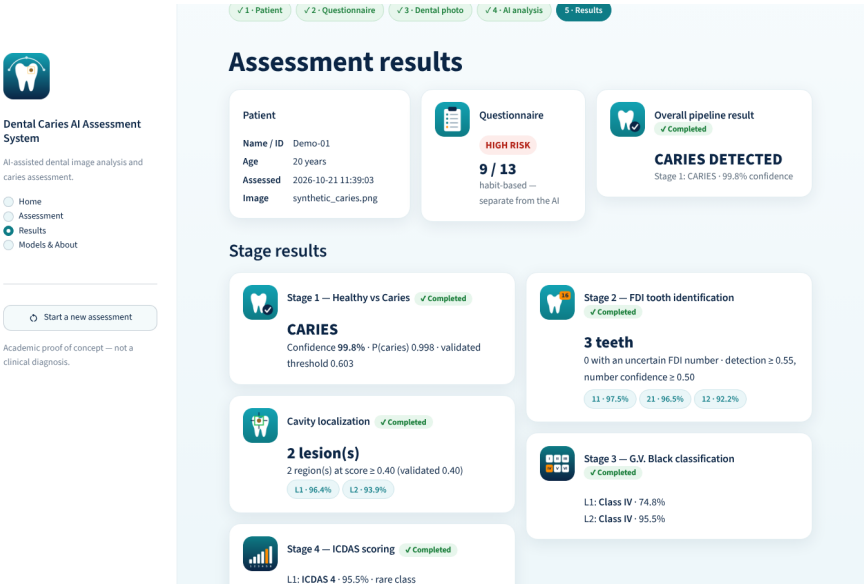
6.5 Upload the photo

Drag a JPG/JPEG/PNG intraoral photo onto the upload box. The preview shows the file name, format and dimensions; Remove image clears it, dropping another file replaces it. Then click Analyze Dental Image. A progress list shows each pipeline step as it actually completes.



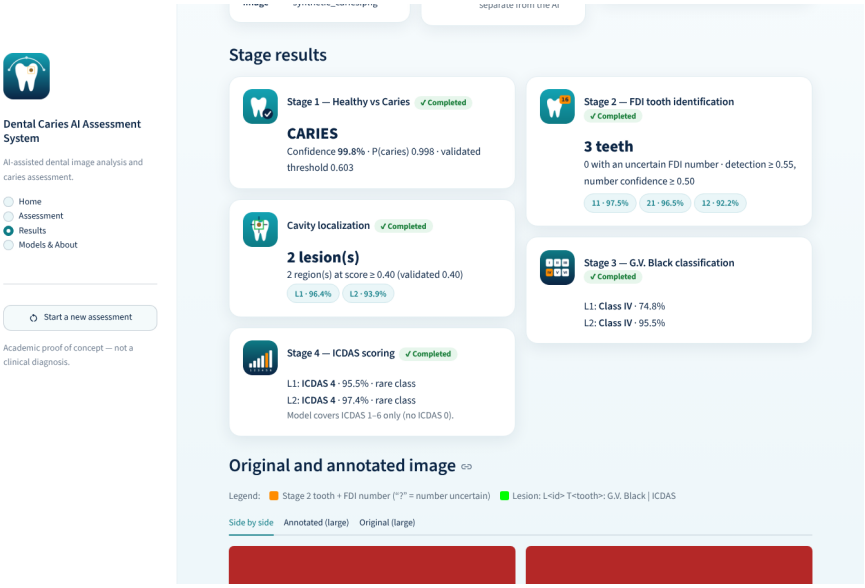
6.6 Results summary

Patient, questionnaire risk and the overall pipeline result.



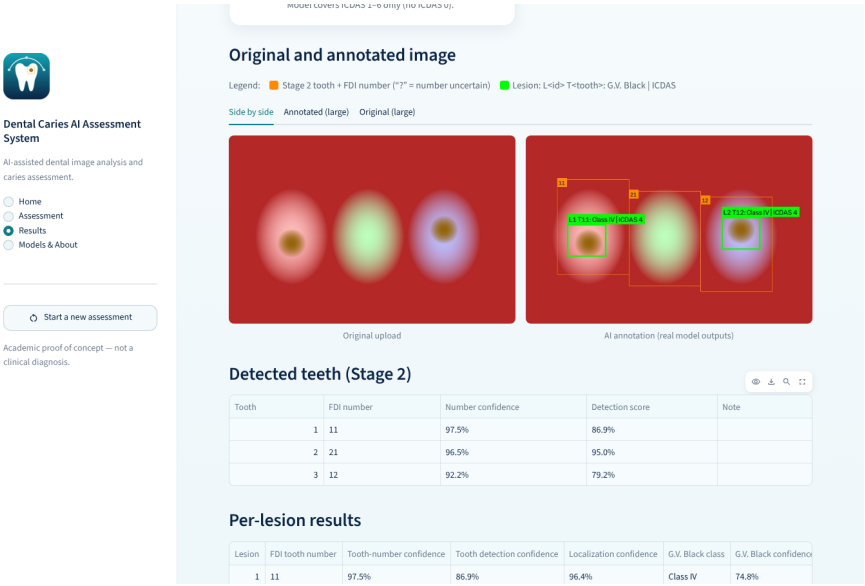
6.7 Stage cards

One card per stage with its output, confidence and a status: Completed, Uncertain, Unavailable, Skipped or Failed.



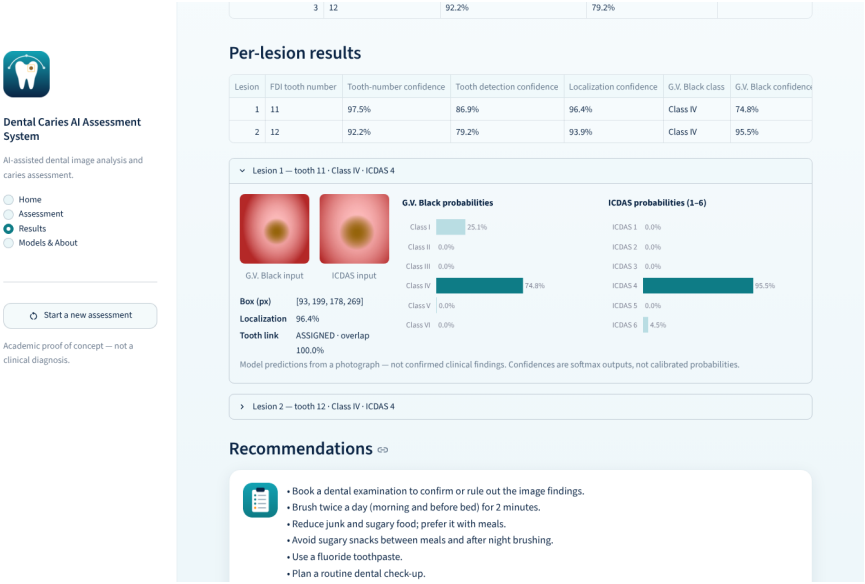
6.8 Images

Original upload next to the annotated image: orange = teeth with FDI number ('?' = uncertain), green = lesions labelled L<id> T<tooth>: G.V. Black | ICDAS. Tabs show each image large (useful on phones).



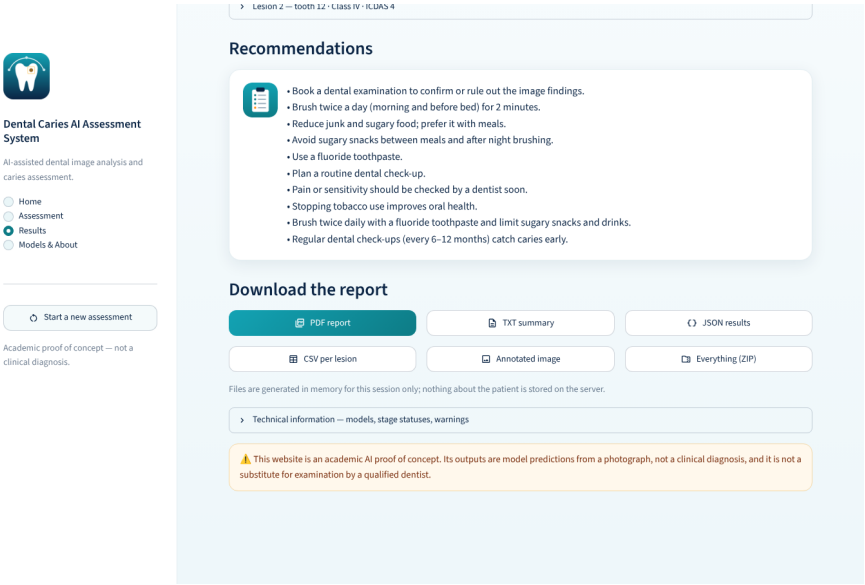
6.9 Per-lesion table

Lesion ID, FDI tooth, tooth confidences, localization confidence, G.V. Black and ICDAS with confidences and uncertainty flags. Expand a lesion to see its model input crops and probability charts.



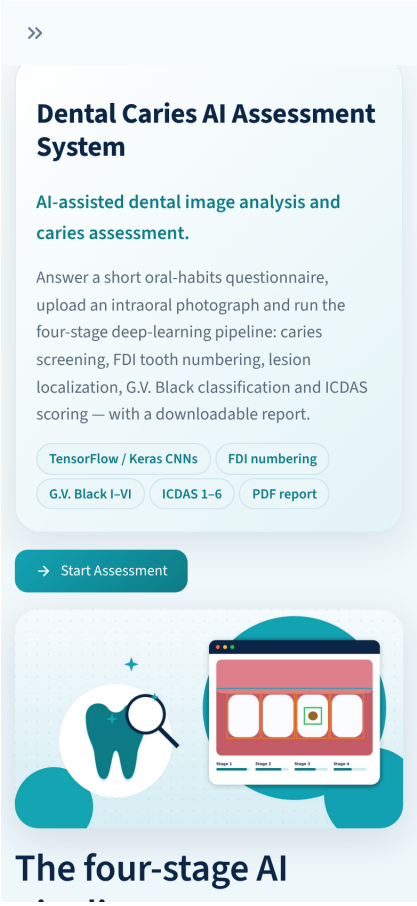
6.10 Downloads

Recommendations and the download buttons (Section 8).



On a phone

The layout adapts to small screens; the menu moves behind the » button.



Home page at phone width.

7. Complete start-to-finish example (reference result)

This example repeats the assessment that your notebook ran in Section 82.1 (output saved in the notebook), so you can check that the website gives the same answer with the same models and the same photo.

1. Find the photo UFV72.jpg (the photo used in the notebook's Section 82.1 example; it comes from your project datasets — search for it inside caries_FDI_dataset_final.zip on Drive) and download it to your computer.
2. Run Steps 1–7 of Section 5 and open the website.
3. Assessment: ID 20, age 20; answers Q1 No, Q2 Once a day, Q3 Sometimes, Q4 Not sure, Q5 6-12 months ago, Q6 Yes, Q7 Yes.
4. Expected questionnaire result: HIGH — 9 / 13 (points 2, 1, 1, 1, 1, 2, 1).
5. Upload UFV72.jpg and click Analyze Dental Image (default options).
6. Compare with the notebook's recorded result below.

Stage	Notebook result for UFV72.jpg (recorded 19 Oct 2026)
Stage 1	CARIES — confidence 100.0% (P(caries) 1.000, threshold 0.603)
Stage 2	3 teeth: 11 (number 100.0%, detection 82.5%), Unknown / uncertain — best guess 11 (83.4%, 69.6%), 21 (100.0%, 73.6%)
Localization	2 regions at score ≥ 0.40 (validated 0.40); 2 valid lesion crops
Lesion 1	tooth 21 · localization 65.6% · G.V. Black Class IV 82.8% · ICDAS 5 60.0%
Lesion 2	tooth Unknown / uncertain (best guess 11) · localization 53.4% · Class IV 100.0% · ICDAS 5 57.9%

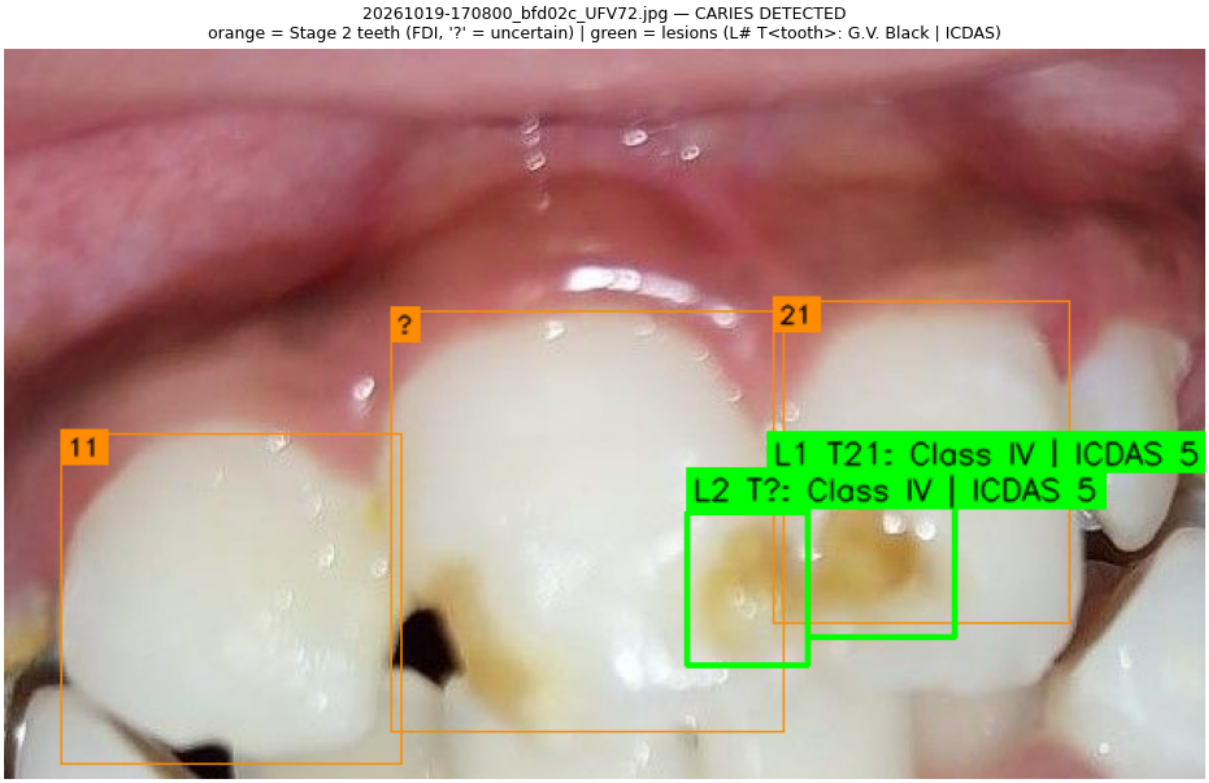


Figure 5 — Real annotated output of your notebook for UFV72.jpg (orange = teeth/FDI, green = lesions). The website draws the same boxes and labels with the same code.

Tip: The website's pipeline was tested to produce identical results to the notebook's own functions (Sections 6.1, 6.6, 6.7, 76, 82) on the same models and images. Small differences can only come from a different file (e.g. a re-saved or resized copy of the photo), a different TensorFlow version or GPU vs CPU rounding (normally < 0.1 %).

8. Download the final report

Button	File	Contents
PDF report	dental_assessment___report.pdf	Professional report: summary, questionnaire, stage table, images, teeth, lesions, advice, limitations, model info
TXT summary	..._summary.txt	The notebook's FINAL DENTAL REPORT text (sections 1–10)
JSON results	..._results.json	All structured results, stage statuses, pipeline steps, model information
CSV per lesion	..._lesions.csv	One row per lesion with confidences and uncertainty flags
Annotated image	..._annotated.jpg	The photo with tooth and lesion boxes
Everything (ZIP)	..._all_results.zip	All five files above

Files download straight to your computer through the browser (the Downloads folder). They are generated in memory and are not saved in Colab or Drive. To keep a copy in Drive, upload the downloaded files to a Drive folder.

9. After a disconnect — restart and keep your work

- Colab sessions end after inactivity or after some hours; the local disk (/content) is then wiped. Your ZIP and models are safe in Drive; only the running website stops.
- To restart: Runtime → Run all is fine, but you only need Steps 1–7 (choose one Option in Step 7). Then open the new link — old links stop working.
- If only the website stopped (Step 8 says “NOT running”) while the session is alive: run Step 6 and Step 7 again.
- Keep the setup notebook in Drive: File → Save a copy in Drive (it is stored in My Drive/Colab Notebooks).
- To update the website, replace Dental_Caries_AI_Website.zip in Drive and run Steps 3–7 again.
- Download reports before closing the browser tab — an assessment lives only in that browser session.

10. Troubleshooting

Problem	What you see	Fix
Drive permission	Step 1 pop-up blocked, or MyDrive missing	Allow pop-ups for colab.research.google.com, run Step 1 again and accept with the same account as Colab. If needed: drive.flush_and_unmount(), then Step 1.
Wrong account / shared folder	Step 2: model folder NOT FOUND	Check the account (Section 3). For a shared folder add the shortcut to My Drive (Section 4), then rerun Step 2.
Missing files	Step 2 lists fewer than 5 .keras files, or Step 5 shows MISSING	Open dentalcaries/pipeline_stage1_to_4_corrected in Drive and check the 10 file names in Section 1. Names must not change (no “(1)”). Stage 2 (FDI) is optional; the others are required.
ZIP not found / incomplete extraction	Step 2: ZIP NOT FOUND; Step 3: “ZIP is damaged” or MISSING lines	Wait for the Drive upload to finish, check the exact file name, upload again if damaged, rerun Steps 2–3.
Dependency conflict	pip warnings in Step 4	Warnings about other Colab packages are harmless. Never install another TensorFlow. If imports fail: Runtime → Restart session, then Steps 1–5.
Model loading error	Step 5: “model file could not be loaded (… deserialize …)”	Keras in Colab is older/newer than the saving version (Keras 3.13.2). Run !pip install -q tensorflow==2.21.0 keras==3.13.2, Runtime → Restart session, then Steps 1–6. A truncated copy: delete the models folder and rerun Step 5.
Missing metadata	Step 5: “metadata file … not found”	Copy the matching *_metadata.json into the Drive model folder; the website refuses to guess thresholds or class mappings.
Port in use / app not starting	Step 6: “did not start” or address already in use	Run Step 9, then Step 6. Read the log printed by Step 8.
Proxy page blank	Option A link loads forever or reconnects	Use the same browser and account as Colab, allow third-party cookies for *.googleusercontent.com, or use Option B.
Tunnel error	Option B: no URL / 502 / 1033	Make sure Step 6 says running, rerun Step 7B. A new run gives a new URL. If GitHub download fails, retry later or use Option A.
Upload fails (403)	Red error after choosing a photo	Start Streamlit only with the Step 6 cell (it disables XSRF for proxied access); reload the page.
Session disconnected	“Runtime disconnected”, link dead	Reconnect and run Steps 1–7 (Section 9).
Not enough RAM / GPU memory	Session crashes, “Your session crashed after using all available RAM”, ResourceExhausted	Runtime → Disconnect and delete runtime, close other Colab tabs, run again. All five models need ≈ 2–3 GB RAM. Try a CPU runtime if the GPU is full.
Very slow analysis	first analysis takes minutes	The first click loads the models (≈ 1–2 min); later analyses take seconds. Use a T4 GPU runtime.
Stage shows Unavailable	result card grey “Unavailable”	The model or its metadata is missing/unloadable — see Models & About in the website and fix the file. This is by design: no fake results.

11. Final checklist

☐	Colab and Drive use the same Google account.
☐	My Drive contains dentalcaries (own folder or shortcut).
☐	Dental_Caries_AI_Website.zip is in My Drive/dentalcaries (or My Drive).
☐	pipeline_stage1_to_4_corrected contains 5 .keras files + 5 *_metadata.json files.
☐	Runtime type is T4 GPU (or CPU, slower).
☐	Step 2 found the ZIP and the model folder.
☐	Step 3 printed OK for all five checked files.
☐	Step 4 printed the TensorFlow, Keras, Streamlit and ReportLab versions.
☐	Step 5 ended with “RESULT: all required models loaded”.
☐	Step 6 printed “Streamlit is running on port 8501”.
☐	Step 7 gave a link and the home page opens.
☐	The questionnaire gives the expected score (example: 9 / 13 HIGH).
☐	An analysis completes and the stage cards show their statuses.
☐	The PDF report downloads and opens.
☐	Step 9 was run (especially after using a public tunnel).

12. Appendix — what was tested

- Automated tests (44, all passing): questionnaire scoring for all 648 answer combinations; upload validation (empty, wrong type, fake content, truncated JPEG, too small, EXIF rotation); model loading from .keras files; Stage 1 stop rules (HEALTHY, UNCERTAIN, continue-if-uncertain); FDI association (assigned, ambiguous, no tooth); no-lesion and missing/corrupted model/metadata behaviour; class mappings taken from metadata; PDF/TXT/JSON/CSV/JPEG/ZIP generation; Streamlit pages (headless AppTest).
- Notebook parity: the notebook's own code (Sections 3, 6.1, 6.6, 6.7, 76, 82, executed from the reference copy) and the website return identical results, annotations, tables and report text on the same models and images.
- Browser end-to-end: the real Streamlit server was started and driven in Chromium: questionnaire → upload → analyse → results → PDF download.
- Not verified here: the website could not be run inside Google Colab itself, and the trained weights could not be downloaded into the test environment (Google Drive downloads were blocked by its network policy). Tests therefore used stand-in models with the same file format, shapes and metadata keys. Run Step 5 (model check) and the Section 7 example in your own Colab session to confirm the real models. The README explains how to re-run the parity test with your trained models.

This website is an academic AI proof of concept. Its outputs are model predictions from a photograph, not a clinical diagnosis, and it is not a substitute for examination by a qualified dentist.